Experiment 8 – Cloud Security Audit

Student Details

Field	Details
Name	Adesh Patankar
Class / Division	TY-BDCE
Subject	Cloud Data Engineering
Experiment	Experiment No. 08 (Case Study)
Date of Submission	27 October 2026

Problem Statement

Perform a security audit of a cloud-based data engineering environment, including access control, encryption and compliance. Identify potential security risks in cloud storage, data processing services, identity and access management and data pipelines. Recommend appropriate controls and document the audit findings.

Aim: To perform a security audit of a cloud-based data engineering environment by evaluating access control, encryption and compliance-related security controls, and to recommend corrective actions.

Case study: ShopNova Online Retail, a fictional e-commerce company whose cloud data platform stores customer data, runs a daily sales ETL pipeline and loads data into an analytics warehouse. All data used is fictional; no real cloud account was audited.

Tools Used and Project Files

The project uses Python 3 (standard library only) and JSON. No paid services, API keys or cloud accounts are needed.

File	Purpose
`main.py`	The audit program: runs all checks and prints and saves the report

File	Purpose
`sample_environment.json`	ShopNova case study: fictional cloud configuration with secure and insecure settings
`test_aws_environment.json`	Second fictional AWS-style environment used to test the tool
`validate_aws_test.py`	Automatic test that compares expected and actual audit results
`reports/`	Timestamped JSON audit reports (created automatically)
`README.md`	Objective, methodology, how to run, limitations

What I Did

1. **Designed the case study environment.** Created a fictional ShopNova cloud setup in JSON with 5 IAM users, 4 roles, 3 storage buckets, a data warehouse, an ETL pipeline, and logging and key-management settings. Good and bad settings were mixed on purpose.
2. **Built the audit program (`main.py`).** It reads the JSON configuration and runs three groups of checks:
   - **Access control:** unapproved admin rights, wildcard (`*`) permissions, MFA status, enabled accounts unused for more than 90 days.
   - **Encryption:** encryption at rest, encryption in transit (TLS), customer-managed keys for sensitive data.
   - **Compliance:** a 10-control checklist (CTL-01 to CTL-10) inspired by ISO 27001 and CIS benchmarks, covering MFA, least privilege, encryption, key rotation, audit logging, log retention, alerts, public storage and versioning. This is a simplified educational checklist and does not certify compliance.
3. **Classified the risks.** Every failed check is rated HIGH, MEDIUM or LOW, with a short reason and a recommended fix.
4. **Generated the report.** The program prints a summary on screen and saves a timestamped JSON report in `reports/` as audit evidence.
5. **Ran the audit on ShopNova** with `python main.py`.
6. **Validated the tool on a second environment.** Created a fictional AWS-style setup (S3 buckets, Redshift, ETL job, two IAM users) with 7 planted problems and ran `python main.py test_aws_environment.json aws_environment_test_report.json`.

7. **Automated the validation.** Wrote `validate_aws_test.py`, which runs the audit and checks that every planted problem is detected and every secure item passes.
8. **Fixed issues found during testing.** Service accounts are no longer asked for MFA; a scoped path like `bucket/folder/*` is no longer wrongly treated as full access; existing reports are never overwritten.

Results

The ShopNova environment was rated **AT RISK**, and the AWS-style test confirmed the tool detects every planted problem.

Measure	ShopNova case study	AWS-style test environment
Total checks	45	31
Passed	27	16
Failed	18	15
High / Medium / Low risk	11 / 6 / 1	10 / 4 / 1
Pass rate	60.0%	51.6%
Overall status	AT RISK	AT RISK

Key findings in ShopNova

- `bob.dataengineer` has admin rights without approval and no MFA (HIGH).
- The `DataEngineer` role grants wildcard storage permissions on all resources (HIGH).
- The `marketing-exports` bucket holds customer emails but is public and unencrypted (HIGH).
- The `daily-sales-etl` pipeline moves customer data without TLS (HIGH).
- `dave.contractor` is still enabled after 227 days of no use and has no MFA (MEDIUM).
- Logs are kept for only 90 days against a required 365, and no security alerts are configured (MEDIUM).

Validation on the AWS-style test environment

Planted problem	Detected
data_analyst has MFA disabled	Yes (MEDIUM)
data_analyst has wildcard permissions	Yes (HIGH)

Planted problem	Detected
customer-raw-data bucket is public	Yes (HIGH)
customer-raw-data not encrypted at rest	Yes (HIGH)
customer-raw-data not encrypted in transit	Yes (HIGH)
ETL pipeline not encrypted in transit	Yes (HIGH)
Cloud audit logging disabled	Yes (HIGH)

All correctly configured items passed: the least-privilege `etl_service` role, the `processed-data` bucket and the Redshift warehouse. The validation script reported **PASSED – all expected results matched**.

Screenshots

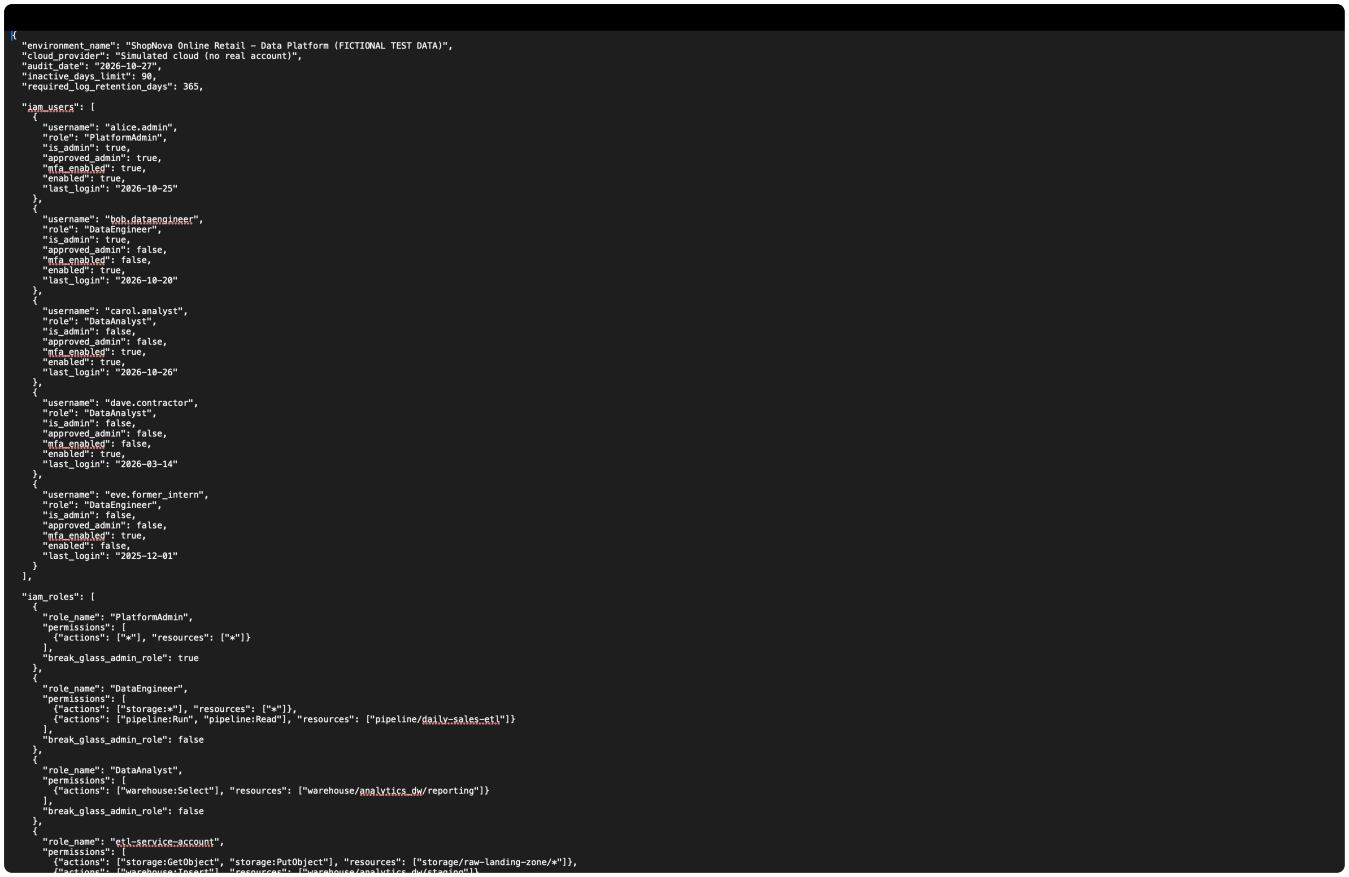
Paste each screenshot under its caption.

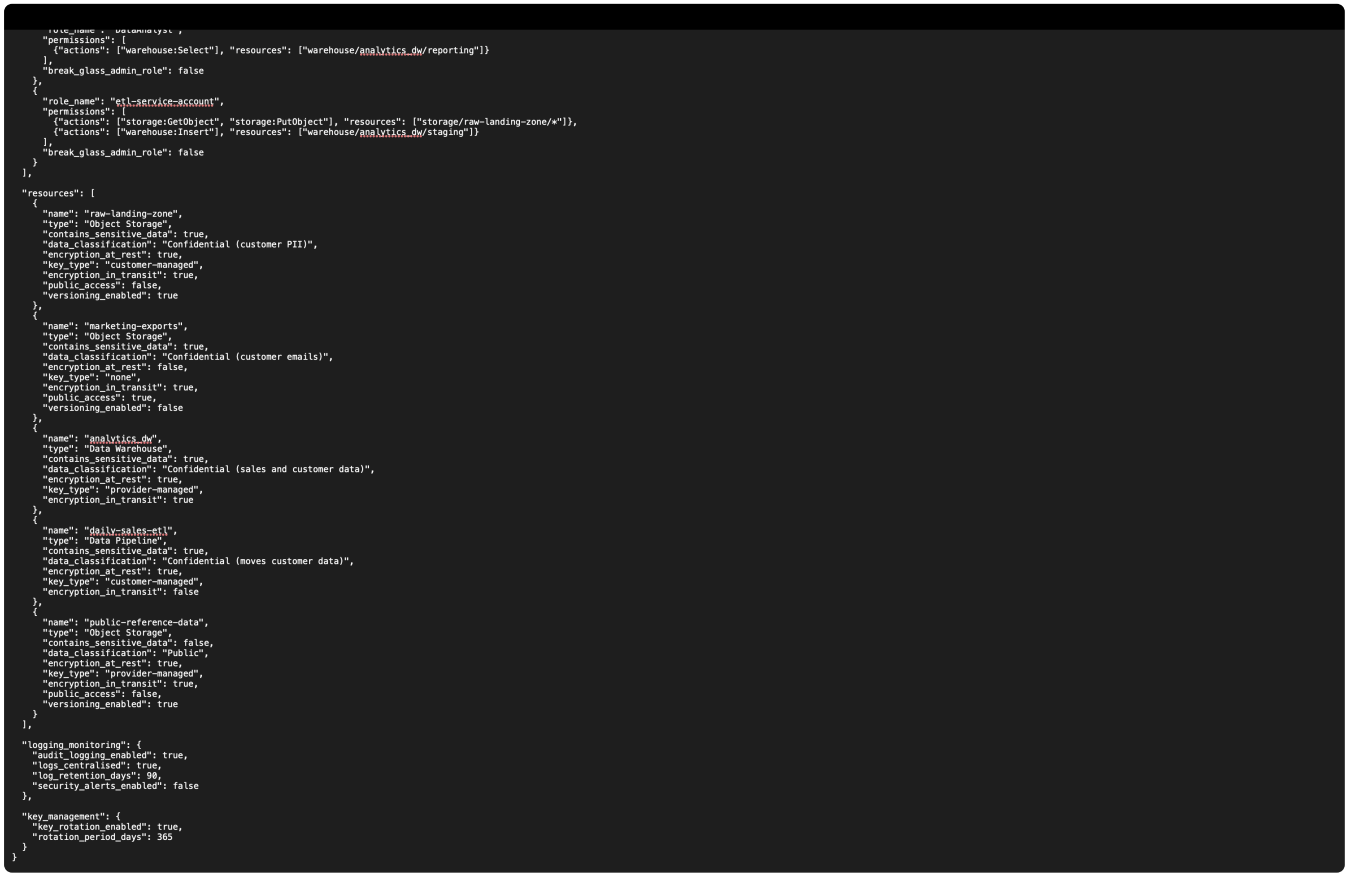
Screenshot 1 – Project files. Terminal inside `Cloud_Security_Audit` after running `ls`.

[paste screenshot here]

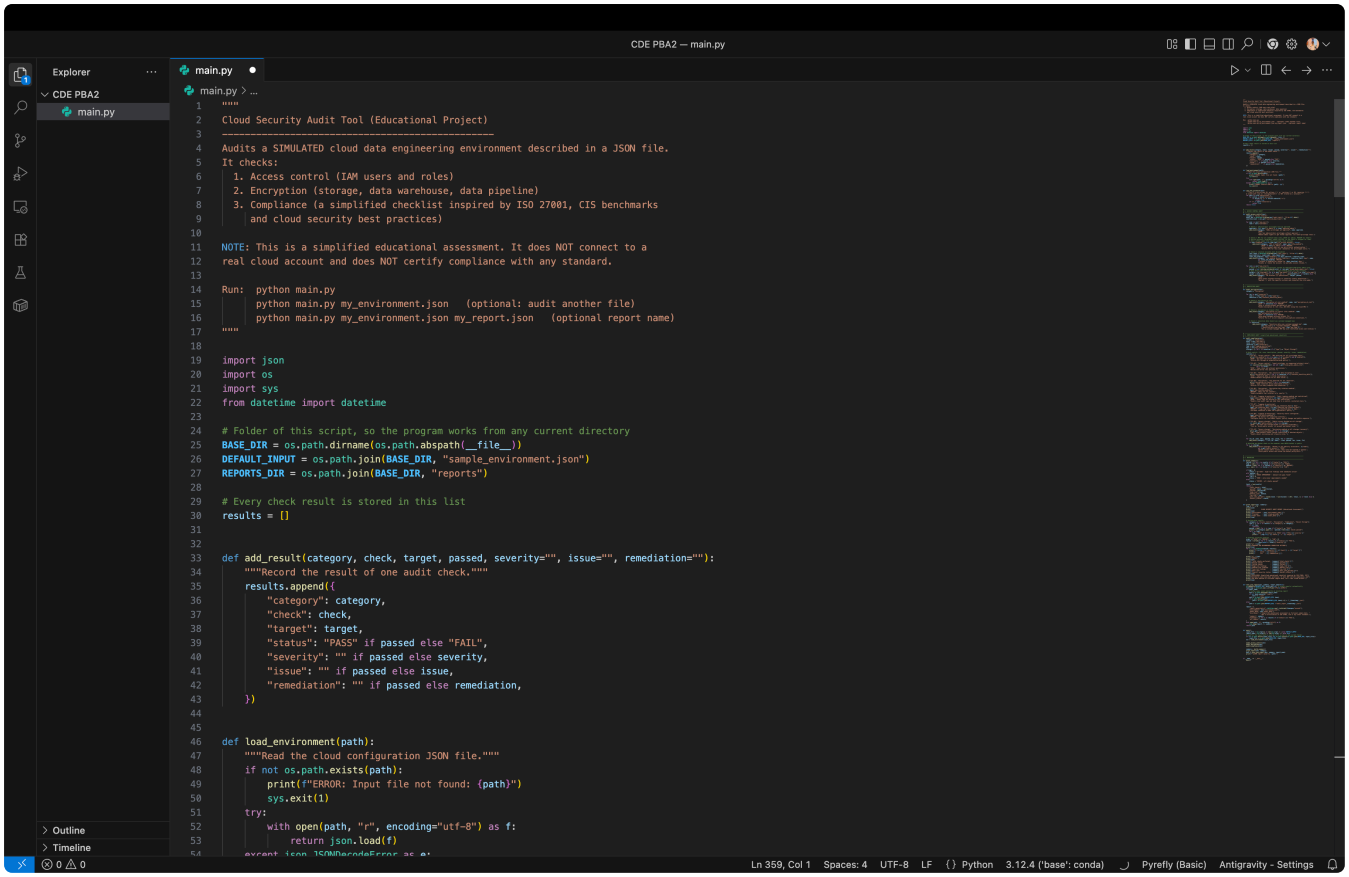


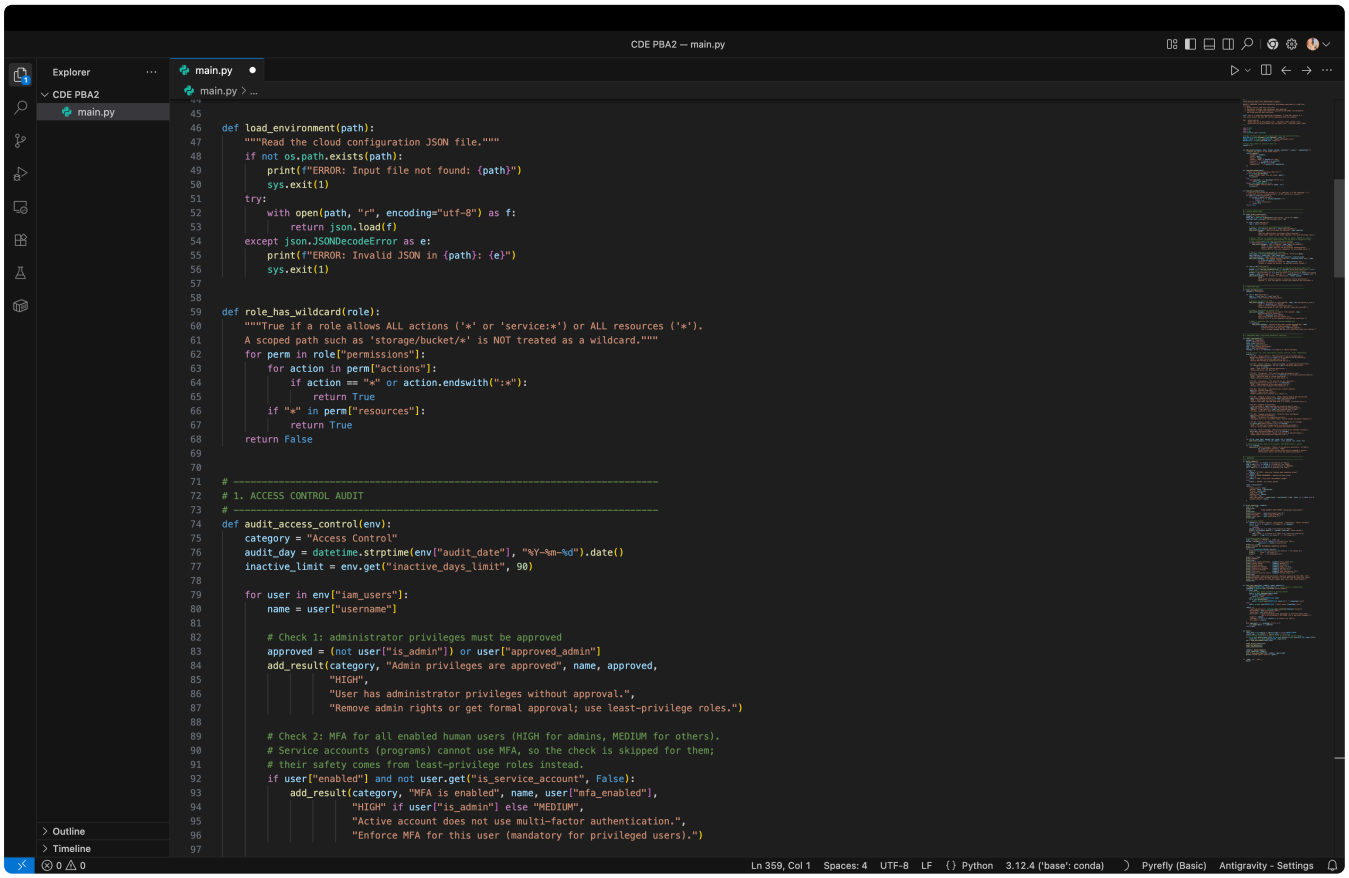
Screenshot 2 – Sample environment (input). `sample_environment.json` open, showing the IAM users and resources.

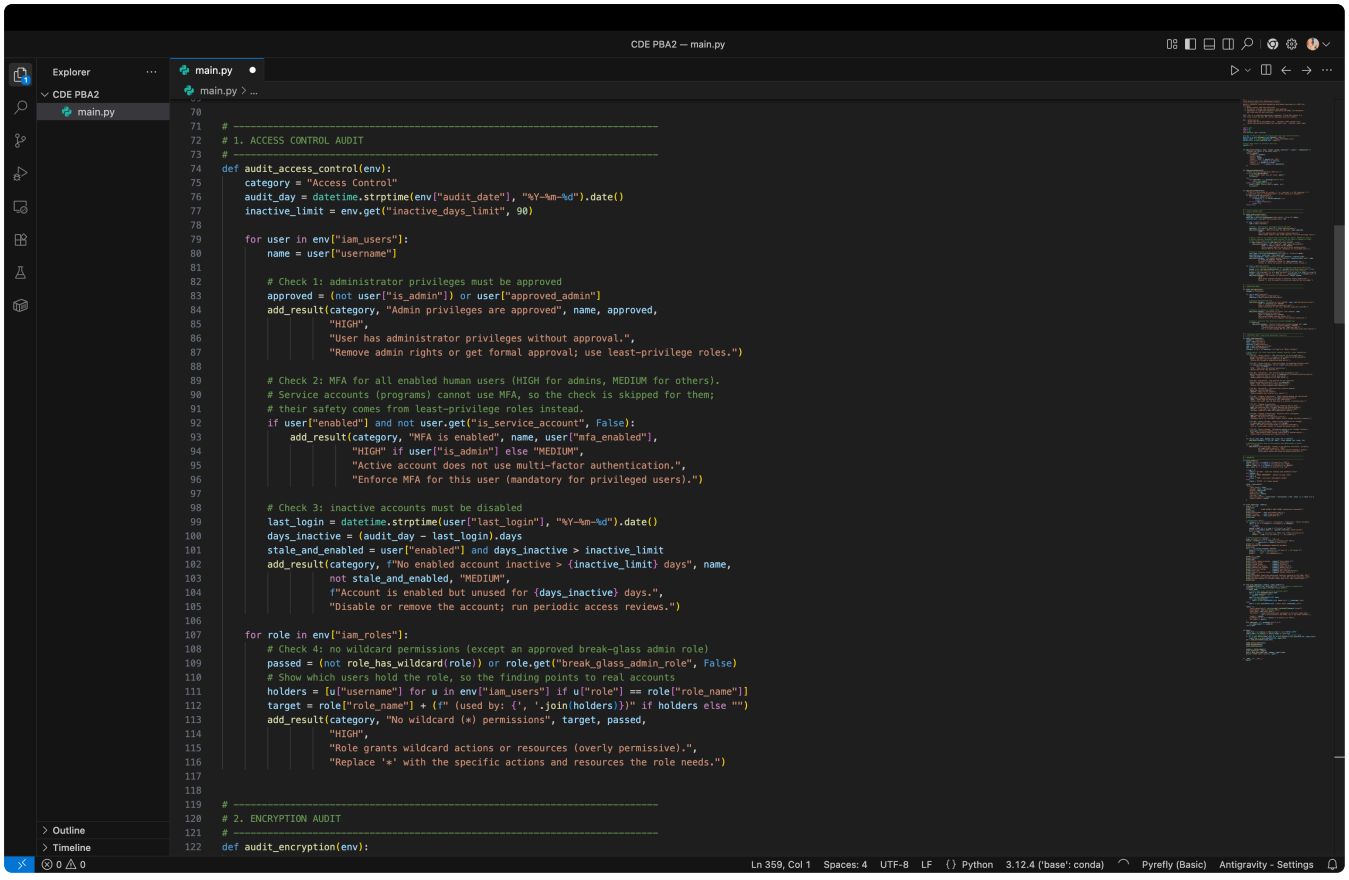


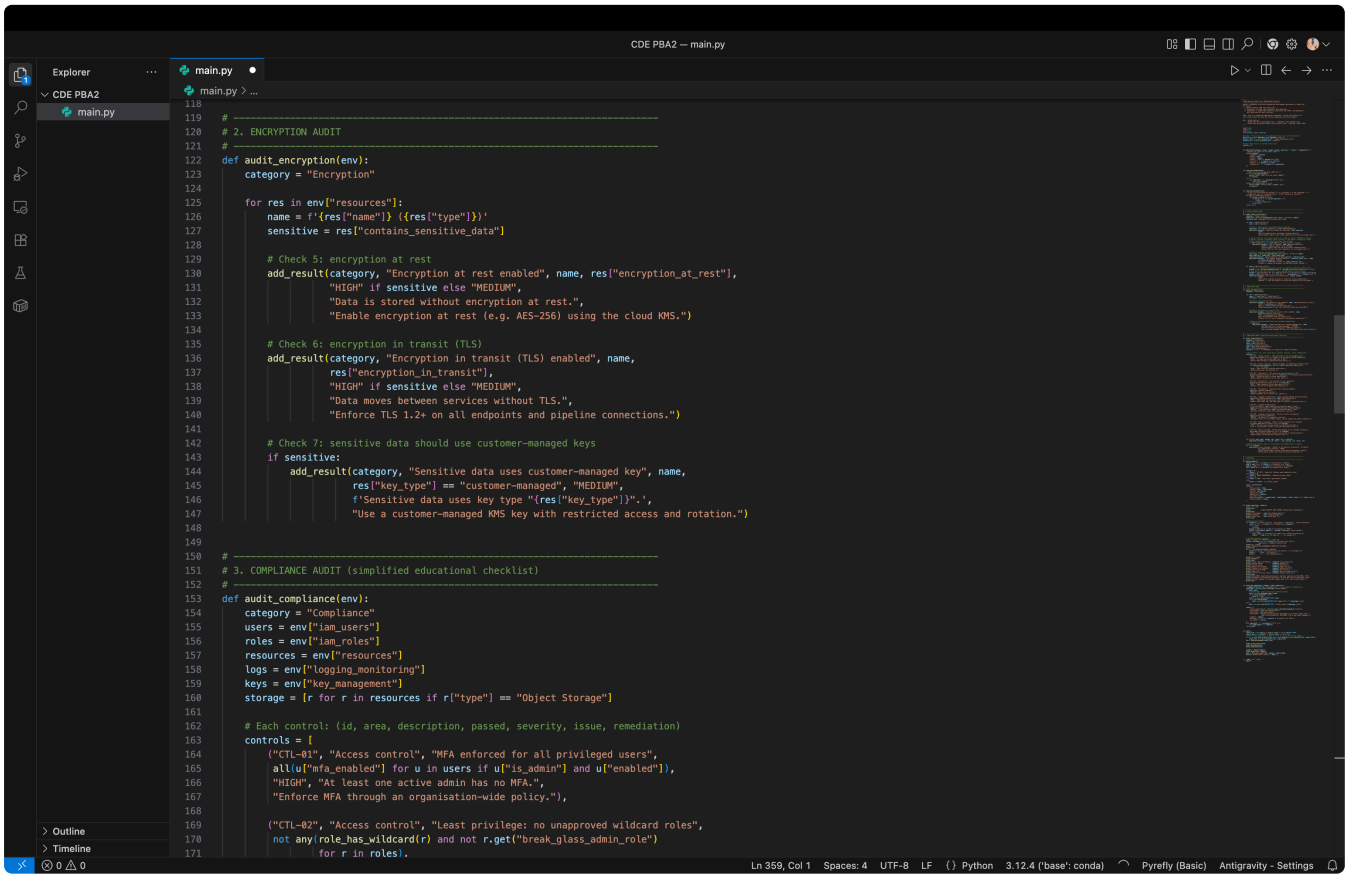


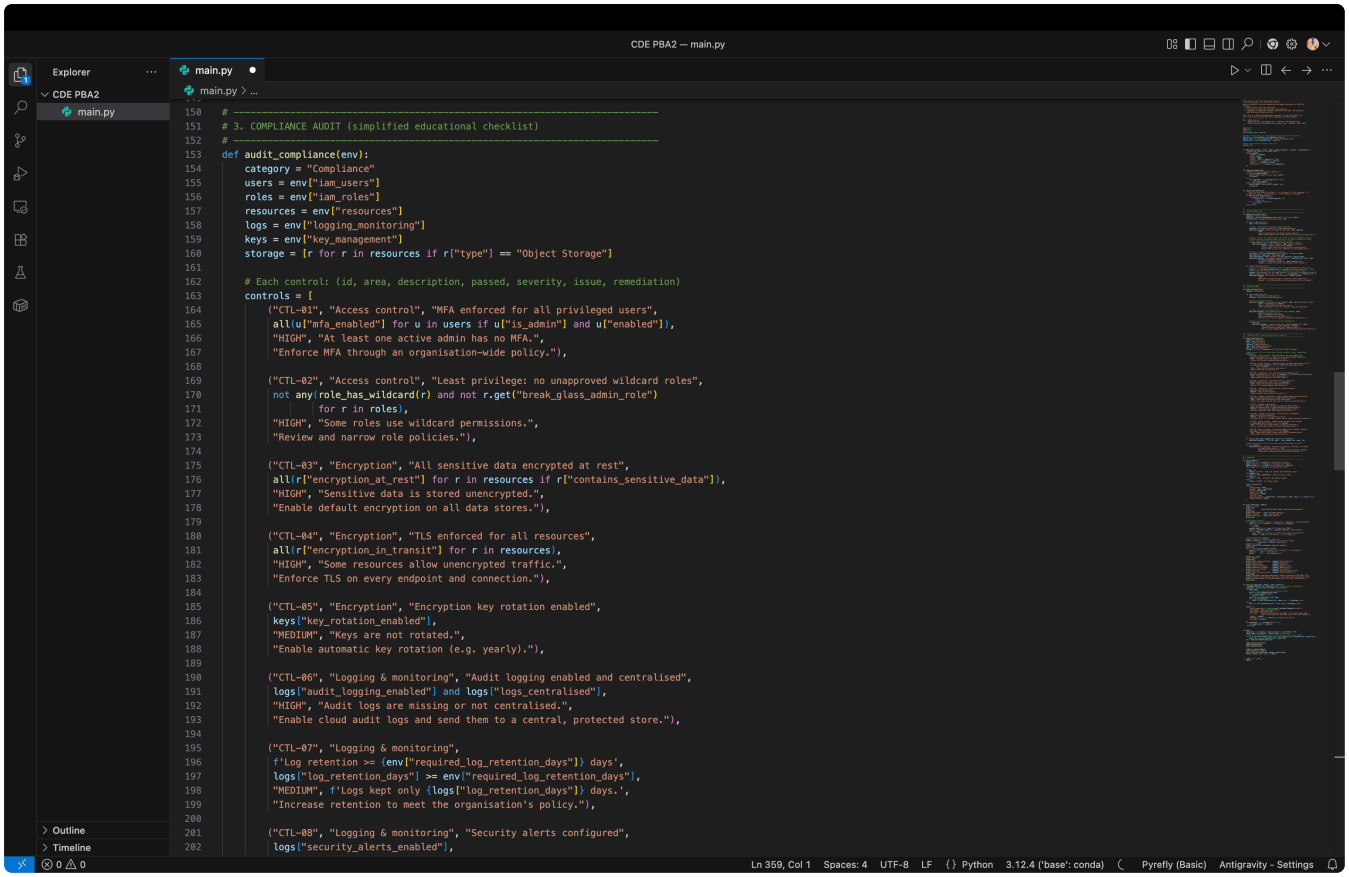
Screenshot 3 – Audit program code. `main.py` open, showing the access control and encryption check functions.

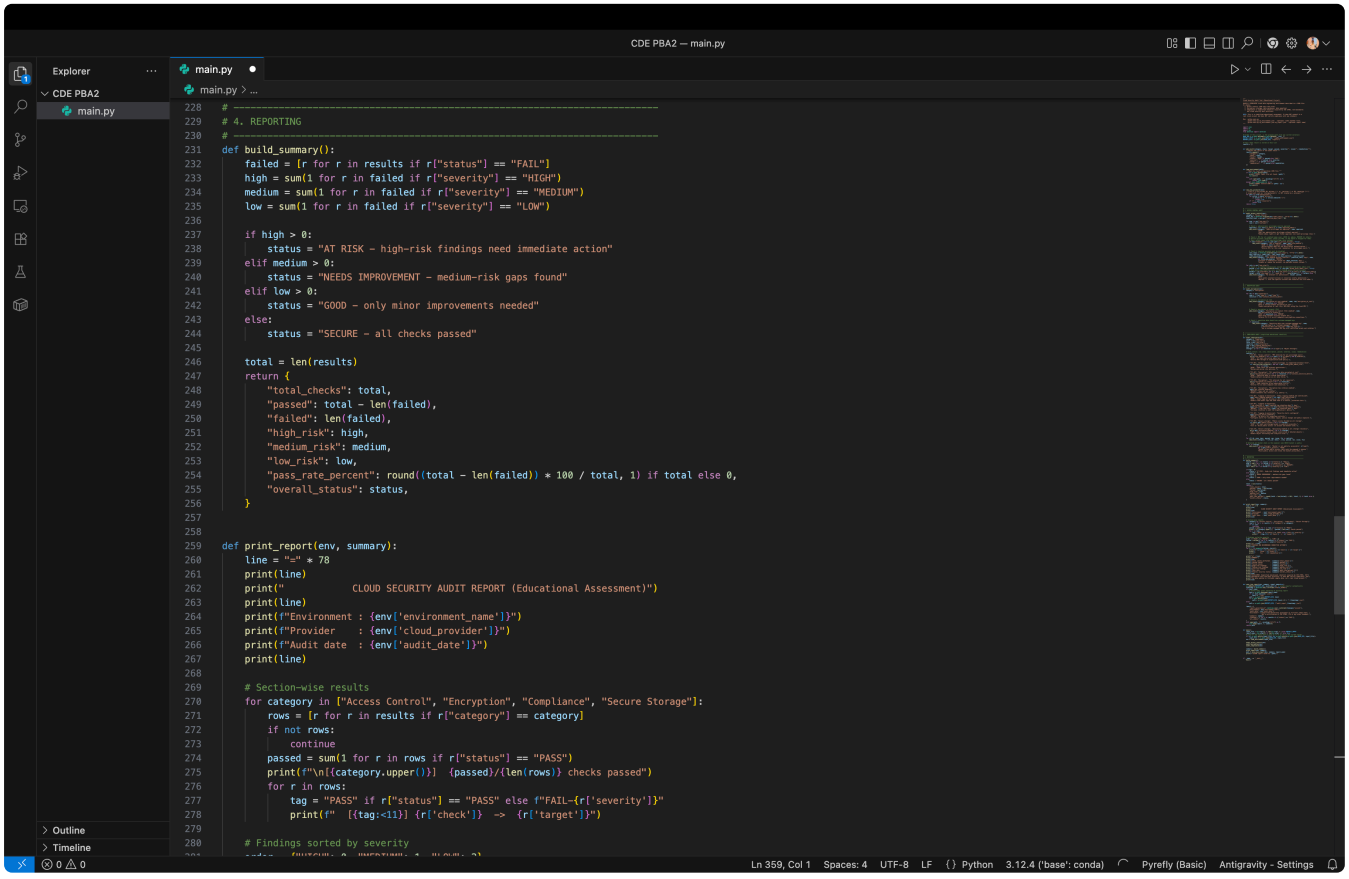

CDE PBA2 — main.py
Explorer
CDE PBA2
main.py
main.py
main.py > ...
"""
Cloud Security Audit Tool (Educational Project)
Audits a SIMULATED cloud data engineering environment described in a JSON file.
It checks:
1. Access control (IAM users and roles)
2. Encryption (storage, data warehouse, data pipeline)
3. Compliance (a simplified checklist inspired by ISO 27001, CIS benchmarks and cloud security best practices)
NOTE: This is a simplified educational assessment. It does NOT connect to a real cloud account and does NOT certify compliance with any standard.
Run: python main.py
python main.py my_environment.json (optional: audit another file)
python main.py my_environment.json my_report.json (optional report name)
"""
import json
import os
import sys
from datetime import datetime
Folder of this script, so the program works from any current directory
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DEFAULT_INPUT = os.path.join(BASE_DIR, "sample_environment.json")
REPORTS_DIR = os.path.join(BASE_DIR, "reports")
Every check result is stored in this list
results = []
def add_result(category, check, target, passed, severity="", issue="", remediation=""):
"""Record the result of one audit check."""
results.append({
"category": category,
"check": check,
"target": target,
"status": "PASS" if passed else "FAIL",
"severity": "" if passed else severity,
"issue": "" if passed else issue,
"remediation": "" if passed else remediation,
})
def load_environment(path):
"""Read the cloud configuration JSON file."""
if not os.path.exists(path):
print(f"ERROR: Input file not found: {path}")
sys.exit(1)
try:
with open(path, "r", encoding="utf-8") as f:
return json.load(f)
Outline
Timeline
Ln 359, Col 1 Spaces: 4 UTF-8 LF Python 3.12.4 ('base': conda) Pyrefly (Basic) Antigravity - Settings

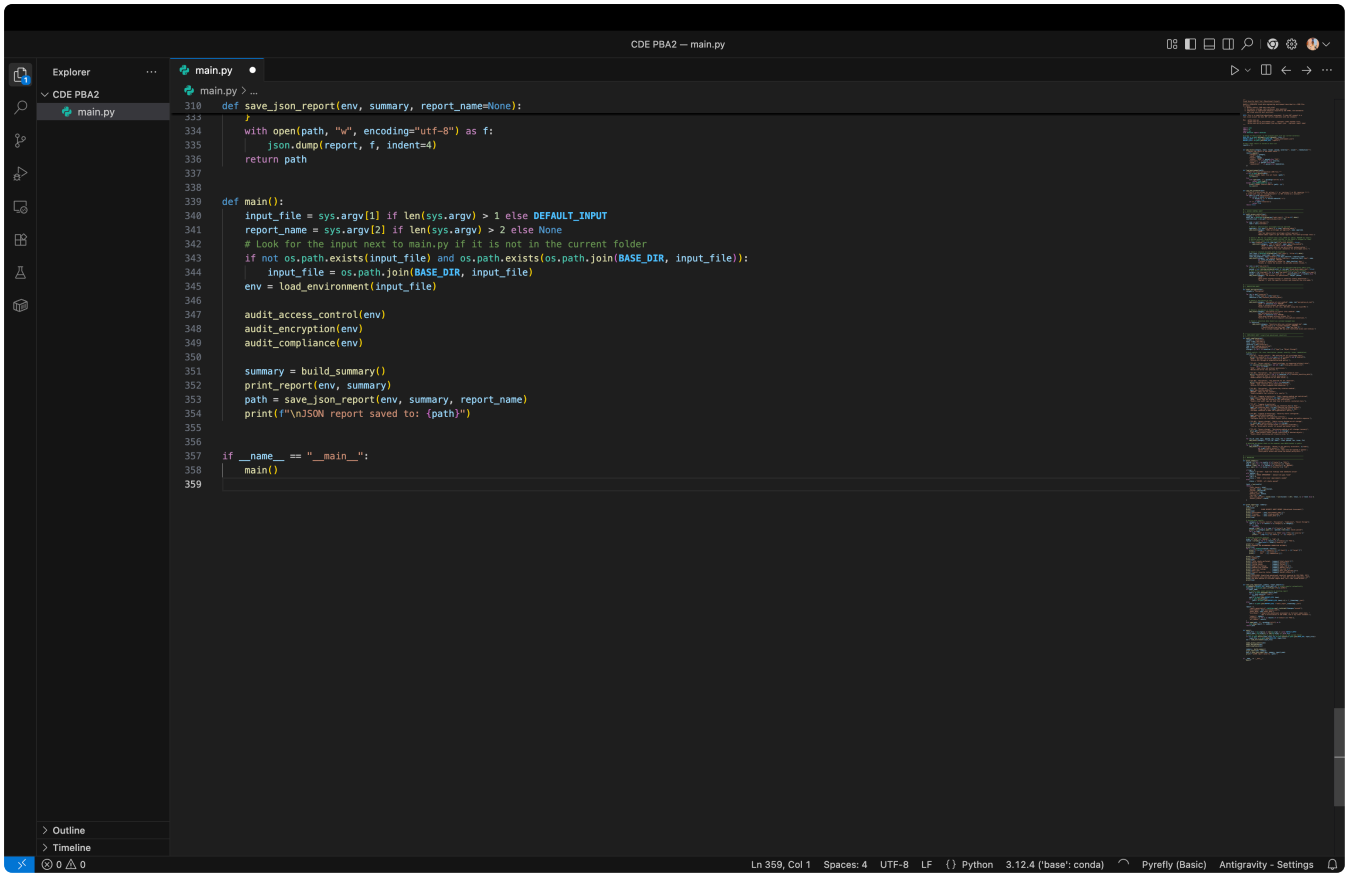

CDE PBA2 — main.py
Explorer
CDE PBA2
main.py
main.py
main.py > ...
def load_environment(path):
"""Read the cloud configuration JSON file."""
if not os.path.exists(path):
print(f"ERROR: Input file not found: {path}")
sys.exit(1)
try:
with open(path, "r", encoding="utf-8") as f:
return json.load(f)
except json.JSONDecodeError as e:
print(f"ERROR: Invalid JSON in {path}: {e}")
sys.exit(1)
def role_has_wildcard(role):
"""True if a role allows ALL actions ('*' or 'service:*') or ALL resources ('*').
A scoped path such as 'storage/bucket/*' is NOT treated as a wildcard."""
for perm in role["permissions"]:
for action in perm["actions"]:
if action == "*" or action.endswith(":*"):
return True
if "*" in perm["resources"]:
return True
return False
---
1. ACCESS CONTROL AUDIT
---
def audit_access_control(env):
category = "Access Control"
audit_day = datetime.strptime(env["audit_date"], "%Y-%m-%d").date()
inactive_limit = env.get("inactive_days_limit", 90)
for user in env["iam_users"]:
name = user["username"]
Check 1: administrator privileges must be approved
approved = (not user["is_admin"]) or user["approved_admin"]
add_result(category, "Admin privileges are approved", name, approved,
"HIGH",
"User has administrator privileges without approval.",
"Remove admin rights or get formal approval; use least-privilege roles.")
Check 2: MFA for all enabled human users (HIGH for admins, MEDIUM for others).
Service accounts (programs) cannot use MFA, so the check is skipped for them;
their safety comes from least-privilege roles instead.
if user["enabled"] and not user.get("is_service_account", False):
add_result(category, "MFA is enabled", name, user["mfa_enabled"],
"HIGH" if user["is_admin"] else "MEDIUM",
"Active account does not use multi-factor authentication.",
"Enforce MFA for this user (mandatory for privileged users).")
Outline
Timeline
Ln 359, Col 1 Spaces: 4 UTF-8 LF Python 3.12.4 ('base': conda) Pyrefly (Basic) Antigravity - Settings


CDE PBA2 — main.py
Explorer
CDE PBA2
main.py
main.py > ...
1. ACCESS CONTROL AUDIT
def audit_access_control(env):
category = "Access Control"
audit_day = datetime.strptime(env["audit_date"], "%Y-%m-%d").date()
inactive_limit = env.get("inactive_days_limit", 90)
for user in env["iam_users"]:
name = user["username"]
Check 1: administrator privileges must be approved
approved = (not user["is_admin"]) or user["approved_admin"]
add_result(category, "Admin privileges are approved", name, approved,
"HIGH",
"User has administrator privileges without approval.",
"Remove admin rights or get formal approval; use least-privilege roles.")
Check 2: MFA for all enabled human users (HIGH for admins, MEDIUM for others).
Service accounts (programs) cannot use MFA, so the check is skipped for them;
their safety comes from least-privilege roles instead.
if user["enabled"] and not user.get("is_service_account", False):
add_result(category, "MFA is enabled", name, user["mfa_enabled"],
"HIGH" if user["is_admin"] else "MEDIUM",
"Active account does not use multi-factor authentication.",
"Enforce MFA for this user (mandatory for privileged users).")
Check 3: inactive accounts must be disabled
last_login = datetime.strptime(user["last_login"], "%Y-%m-%d").date()
days_inactive = (audit_day - last_login).days
stale_and_enabled = user["enabled"] and days_inactive > inactive_limit
add_result(category, f"No enabled account inactive > {inactive_limit} days", name,
not stale_and_enabled, "MEDIUM",
f"Account is enabled but unused for {days_inactive} days.",
"Disable or remove the account; run periodic access reviews.")
for role in env["iam_roles"]:
Check 4: no wildcard permissions (except an approved break-glass admin role)
passed = (not role_has_wildcard(role)) or role.get("break_glass_admin_role", False)
Show which users hold the role, so the finding points to real accounts
holders = [u["username"] for u in env["iam_users"] if u["role"] == role["role_name"]]
target = role["role_name"] + (f" (used by: {', '.join(holders)})" if holders else "")
add_result(category, "No wildcard (*) permissions", target, passed,
"HIGH",
"Role grants wildcard actions or resources (overly permissive).",
"Replace '*' with the specific actions and resources the role needs.")
2. ENCRYPTION AUDIT
def audit_encryption(env):
Outline
Timeline
Ln 359, Col 1 Spaces: 4 UTF-8 LF Python 3.12.4 ('base': conda) Pyrefly (Basic) Antigravity - Settings

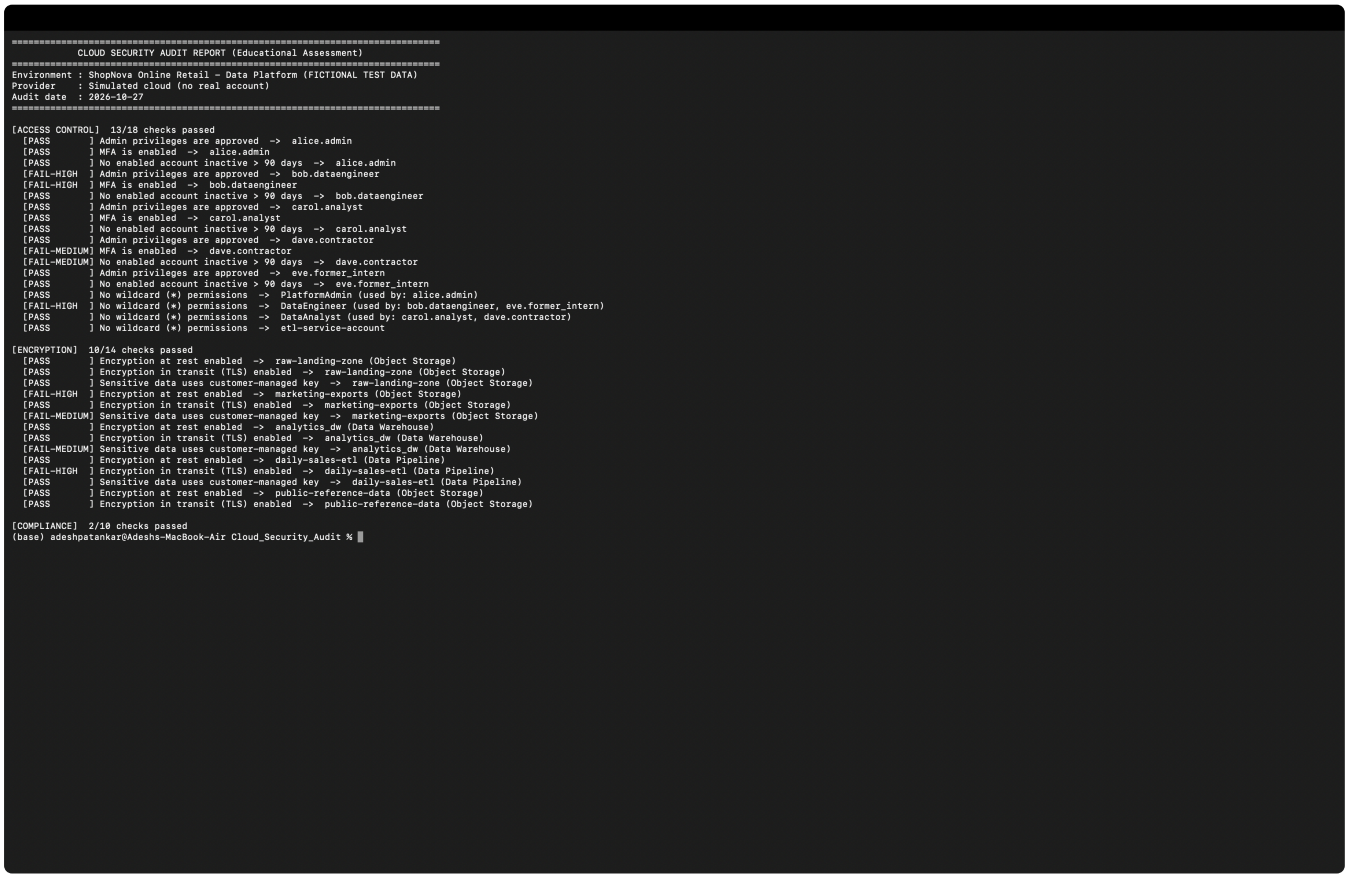

CDE PBA2 — main.py
Explorer
CDE PBA2
main.py
main.py > ...
2. ENCRYPTION AUDIT
def audit_encryption(env):
category = "Encryption"
for res in env["resources"]:
name = f'{res["name"]} ({res["type"]})'
sensitive = res["contains_sensitive_data"]
Check 5: encryption at rest
add_result(category, "Encryption at rest enabled", name, res["encryption_at_rest"],
"HIGH" if sensitive else "MEDIUM",
"Data is stored without encryption at rest.",
"Enable encryption at rest (e.g. AES-256) using the cloud KMS.")
Check 6: encryption in transit (TLS)
add_result(category, "Encryption in transit (TLS) enabled", name,
res["encryption_in_transit"],
"HIGH" if sensitive else "MEDIUM",
"Data moves between services without TLS.",
"Enforce TLS 1.2+ on all endpoints and pipeline connections.")
Check 7: sensitive data should use customer-managed keys
if sensitive:
add_result(category, "Sensitive data uses customer-managed key", name,
res["key_type"] == "customer-managed", "MEDIUM",
f'Sensitive data uses key type "{res["key_type"]}".',
"Use a customer-managed KMS key with restricted access and rotation.")
3. COMPLIANCE AUDIT (simplified educational checklist)
def audit_compliance(env):
category = "Compliance"
users = env["iam_users"]
roles = env["iam_roles"]
resources = env["resources"]
logs = env["logging_monitoring"]
keys = env["key_management"]
storage = [r for r in resources if r["type"] == "Object Storage"]
Each control: (id, area, description, passed, severity, issue, remediation)
controls = [
("CTL-01", "Access control", "MFA enforced for all privileged users",
all(u["mfa_enabled"] for u in users if u["is_admin"] and u["enabled"]),
"HIGH", "At least one active admin has no MFA.",
"Enforce MFA through an organisation-wide policy."),
("CTL-02", "Access control", "Least privilege: no unapproved wildcard roles",
not any(role_has_wildcard(r) and not r.get("break_glass_admin_role")
for r in roles),
Outline
Timeline
Ln 359, Col 1 Spaces: 4 UTF-8 LF Python 3.12.4 ('base': conda) Pyrefly (Basic) Antigravity - Settings


CDE PBA2 — main.py
Explorer
CDE PBA2
main.py
main.py > ...
3. COMPLIANCE AUDIT (simplified educational checklist)
def audit_compliance(env):
category = "Compliance"
users = env["iam_users"]
roles = env["iam_roles"]
resources = env["resources"]
logs = env["logging_monitoring"]
keys = env["key_management"]
storage = [r for r in resources if r["type"] == "Object Storage"]
Each control: (id, area, description, passed, severity, issue, remediation)
controls = [
("CTL-01", "Access control", "MFA enforced for all privileged users",
all(u["mfa_enabled"] for u in users if u["is_admin"] and u["enabled"]),
"HIGH", "At least one active admin has no MFA.",
"Enforce MFA through an organisation-wide policy."),
("CTL-02", "Access control", "Least privilege: no unapproved wildcard roles",
not any(role_has_wildcard(r) and not r.get("break_glass_admin_role")
for r in roles),
"HIGH", "Some roles use wildcard permissions.",
"Review and narrow role policies."),
("CTL-03", "Encryption", "All sensitive data encrypted at rest",
all(r["encryption_at_rest"] for r in resources if r["contains_sensitive_data"]),
"HIGH", "Sensitive data is stored unencrypted.",
"Enable default encryption on all data stores."),
("CTL-04", "Encryption", "TLS enforced for all resources",
all(r["encryption_in_transit"] for r in resources),
"HIGH", "Some resources allow unencrypted traffic.",
"Enforce TLS on every endpoint and connection."),
("CTL-05", "Encryption", "Encryption key rotation enabled",
keys["key_rotation_enabled"],
"MEDIUM", "Keys are not rotated.",
"Enable automatic key rotation (e.g. yearly)."),
("CTL-06", "Logging & monitoring", "Audit logging enabled and centralised",
logs["audit_logging_enabled"] and logs["logs_centralised"],
"HIGH", "Audit logs are missing or not centralised.",
"Enable cloud audit logs and send them to a central, protected store."),
("CTL-07", "Logging & monitoring",
f'Log retention >= {env["required_log_retention_days"]} days',
logs["log_retention_days"] >= env["required_log_retention_days"],
"MEDIUM", f'Logs kept only {logs["log_retention_days"]} days.',
"Increase retention to meet the organisation's policy."),
("CTL-08", "Logging & monitoring", "Security alerts configured",
logs["security_alerts_enabled"],
Outline
Timeline
Ln 359, Col 1 Spaces: 4 UTF-8 LF Python 3.12.4 ('base': conda) Pyrefly (Basic) Antigravity - Settings


CDE PBA2 — main.py
Explorer
CDE PBA2
main.py
main.py > ...
4. REPORTING
def build_summary():
failed = [r for r in results if r["status"] == "FAIL"]
high = sum(1 for r in failed if r["severity"] == "HIGH")
medium = sum(1 for r in failed if r["severity"] == "MEDIUM")
low = sum(1 for r in failed if r["severity"] == "LOW")
if high > 0:
status = "AT RISK – high-risk findings need immediate action"
elif medium > 0:
status = "NEEDS IMPROVEMENT – medium-risk gaps found"
elif low > 0:
status = "GOOD – only minor improvements needed"
else:
status = "SECURE – all checks passed"
total = len(results)
return {
"total_checks": total,
"passed": total - len(failed),
"failed": len(failed),
"high_risk": high,
"medium_risk": medium,
"low_risk": low,
"pass_rate_percent": round((total - len(failed)) * 100 / total, 1) if total else 0,
"overall_status": status,
}
def print_report(env, summary):
line = "=" * 78
print(line)
print("CLOUD SECURITY AUDIT REPORT (Educational Assessment)")
print(line)
print(f"Environment : {env['environment_name']}")
print(f"Provider : {env['cloud_provider']}")
print(f"Audit date : {env['audit_date']}")
print(line)
Section-wise results
for category in ["Access Control", "Encryption", "Compliance", "Secure Storage"]:
rows = [r for r in results if r["category"] == category]
if not rows:
continue
passed = sum(1 for r in rows if r["status"] == "PASS")
print(f"\n[{category.upper()}] {passed}/{len(rows)} checks passed")
for r in rows:
tag = "PASS" if r["status"] == "PASS" else f"FAIL-{r['severity']}"
print(f" [{tag:<11}] {r['check']} -> {r['target']}")
Findings sorted by severity
Outline
Timeline
Ln 359, Col 1 Spaces: 4 UTF-8 LF Python 3.12.4 ('base': conda) Pyrefly (Basic) Antigravity - Settings



Screenshot 4 – Audit output: section-wise checks. Output of `python main.py` showing the PASS/FAIL list for access control and encryption.



Screenshot 5 – Findings and summary. The findings list and the summary block (45 checks, 18 failed, AT RISK).



```
F-12 [MEDIUM] MFA is enabled -> dave.contractor
      Issue : Active account does not use multi-factor authentication.
      Fix   : Enforce MFA for this user (mandatory for privileged users).
F-13 [MEDIUM] No enabled account inactive > 90 days -> dave.contractor
      Issue : Account is enabled but unused for 227 days.
      Fix   : Disable or remove the account; run periodic access reviews.
F-14 [MEDIUM] Sensitive data uses customer-managed key -> marketing-exports (Object Storage)
      Issue : Sensitive data uses key type "none".
      Fix   : Use a customer-managed KMS key with restricted access and rotation.
F-15 [MEDIUM] Sensitive data uses customer-managed key -> analytics_dw (Data Warehouse)
      Issue : Sensitive data uses key type "provider-managed".
      Fix   : Use a customer-managed KMS key with restricted access and rotation.
F-16 [MEDIUM] CTL-07 Log retention >= 365 days -> Logging & monitoring
      Issue : Logs kept only 90 days.
      Fix   : Increase retention to meet the organisation's policy.
F-17 [MEDIUM] CTL-08 Security alerts configured -> Logging & monitoring
      Issue : No alerts for suspicious activity.
      Fix   : Configure alerts for root/admin logins, policy changes and public exposure.
F-18 [LOW] CTL-10 Versioning enabled on all storage (recovery) -> Secure storage
      Issue : Some buckets cannot recover overwritten or deleted objects.
      Fix   : Enable object versioning and lifecycle rules.

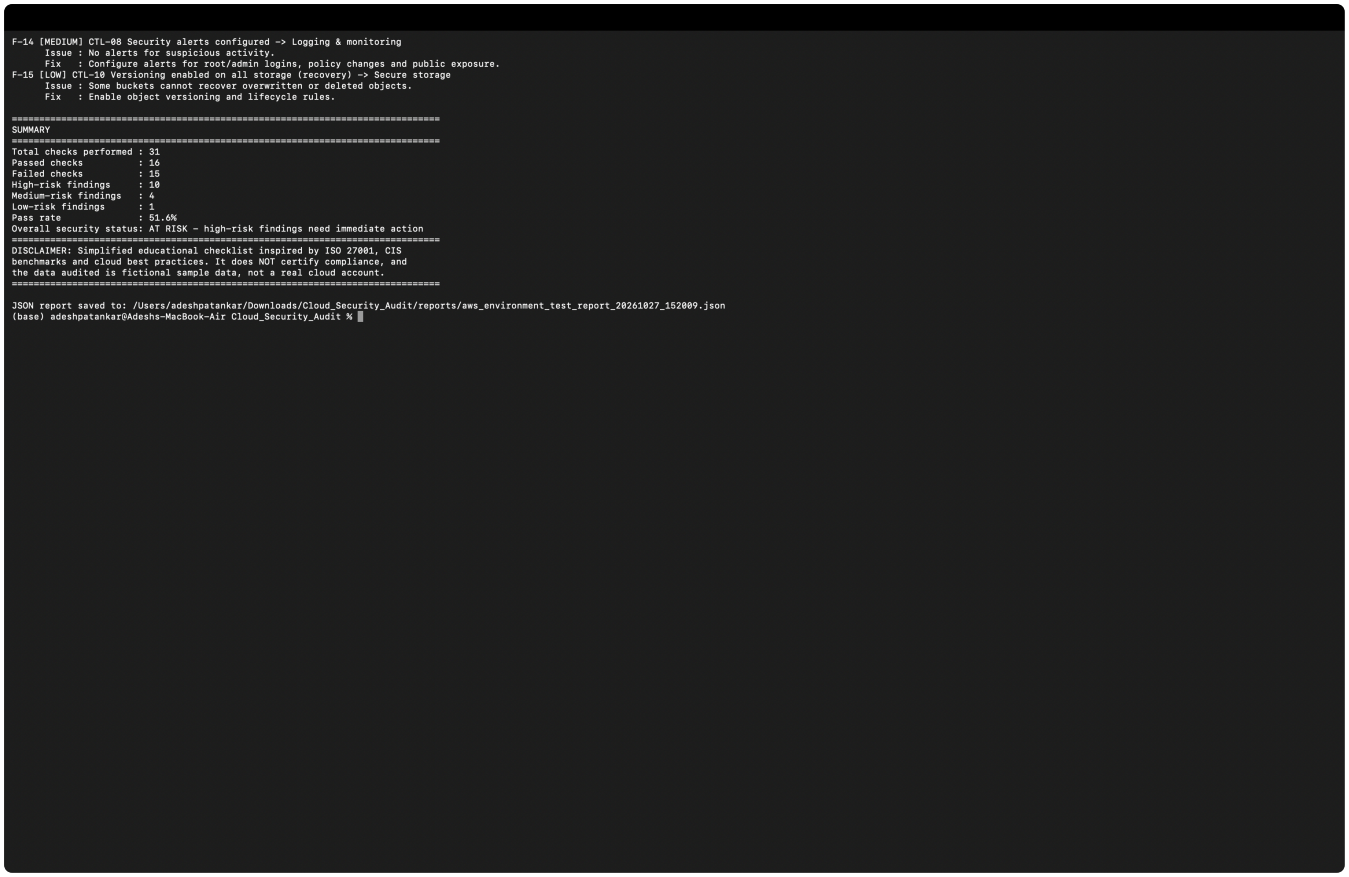
================================================================================
SUMMARY
================================================================================
Total checks performed : 45
Passed checks          : 27
Failed checks          : 18
High-risk findings     : 11
Medium-risk findings   : 6
Low-risk findings      : 1
Pass rate              : 60.0%
Overall security status: AT RISK – high-risk findings need immediate action
================================================================================
DISCLAIMER: Simplified educational checklist inspired by ISO 27001, CIS
benchmarks and cloud best practices. It does NOT certify compliance, and
the data audited is fictional sample data, not a real cloud account.
================================================================================

JSON report saved to: /Users/adeshpatankar/Downloads/Cloud_Security_Audit/reports/audit_report_20261027_151902.json
(base) adeshpatankar@Adeshs-MacBook-Air Cloud_Security_Audit %
```

Screenshot 6 – Generated JSON report. The `reports/` folder or the opened report file.



Screenshot 7 – AWS-style test run. Summary of `python main.py test_aws_environment.json aws_environment_test_report.json` (31 checks, 15 failed).



Screenshot 8 – Validation result. Output of `python validate_aws_test.py` showing the table and "PASSED – all expected results matched".

Image — Screenshot 2026-10-27 at 3.20.38 PM.png

Conclusion

The security audit of the ShopNova cloud data engineering environment was performed successfully using a Python audit tool. It identified weaknesses in access control, encryption, logging and storage configuration, rated each by risk and recommended fixes. Testing on a second AWS-style environment confirmed that the tool detects all planted problems and passes correctly configured resources. Applying least privilege, MFA, encryption at rest and in transit, blocked public access and centralised logging with alerts would reduce these risks. This is a simplified educational assessment on fictional data and does not certify compliance with any standard.